Projeto 2

El proyecto a usar es el de administración de la programación de vigilantes que tenemos en otro chat

Indicaciones:

Entregas semanais com apresentação (16/10, 23/10, 30/10) · **Entrega final:** ZIP no Moodle em **06/11** · **Prova individual** sobre o projeto em **27/11**

Objetivo

Cada grupo escolhe **qualquer domínio de negócio** (clínica veterinária, biblioteca, eventos, oficina mecânica, academia...) e constrói uma aplicação **mais completa que a do Projeto 1**, publicada na AWS (Learner Lab).

Neste projeto **vocês vão usar IA (GitHub Copilot) em todas as etapas**: especificar, modelar a arquitetura, gerar código e documentar. A IA escreve, **mas o grupo decide e responde por tudo**. Em qualquer apresentação, qualquer membro pode ser chamado a explicar qualquer parte do sistema. A prova de 27/11 é individual e é sobre o projeto do *seu* grupo.

Como vamos trabalhar: especificação antes do código

Vocês não começam pelo código. A sequência é:

1. **Especificar** (specs/spec.md): qual problema, para quem, quais histórias de usuário e como saber se cada uma está pronta (critérios de aceite).
2. **Arquitetar** (specs/design.md): diagrama, modelo de dados, rotas da API, decisões justificadas (ADRs).
3. **Planejar** (specs/tasks.md): tarefas pequenas, cada uma ligada a um requisito, com responsável.
4. **Implementar**: uma tarefa por vez, dando ao Copilot a spec e o design como contexto.

Esses arquivos são o **contexto** que vocês dão ao Copilot. Quanto melhor a especificação, melhor o código gerado.

Requisitos mínimos (valem para qualquer domínio)

ID	Requisito
RF-01	CRUD completo de **pelo menos 2 entidades relacionadas** (1:N ou N:N) persistidas no **Amazon RDS**
RF-02	**Upload de arquivo** (imagem, PDF, CSV...) ligado a uma entidade e armazenado no **S3** via **URL pré-assinada** gerada pelo backend
RF-03	Um arquivo novo no S3 **dispara uma Lambda** que faz algo útil (miniatura, extrair metadados, importar CSV...) e grava o resultado no RDS

ID	Requisito
RF-04	Um **relatório** (ex.: GET /relatorios/...) exposto pelo **API Gateway** e implementado em **Lambda**, com consulta usando JOIN/GROUP BY
RF-05	**Front-end web** publicado (ex.: S3 static website ou EC2 + Docker) consumindo a API
RNF-01	**RDS em subnet privada**, sem acesso público. Security Groups documentados
RNF-02	**Nenhuma senha ou chave no repositório**
RNF-03	O sistema volta a funcionar numa nova sessão do Learner Lab seguindo o README (sem IP fixo no código)
RNF-04	Decisão justificada (ADR): o CRUD roda em **Lambda** ou em **container (EC2/ECS)**?
RNF-05	**Não usar IaC** (CloudFormation/Terraform/SAM). Crie os recursos pelo console/CLI e documente no README

Bônus: uso justificado de EFS ou EBS; CI/CD com GitHub Actions; Secrets Manager/Parameter Store; CloudFront; alarmes no CloudWatch.

Atenção ao Learner Lab

- Não dá para criar roles IAM: Lambdas usam a **LabRole** e EC2 usa o **LabInstanceProfile**.
- A sessão expira, e ao reiniciar o **IP público da EC2 muda**. Não fixe IPs no código.
- O orçamento é limitado: **não deixe NAT Gateway ligado** e use instâncias pequenas (ex.: db.t3.micro, t3.micro).
- Lambda dentro da VPC (para falar com o RDS) **não tem internet**. Para acessar o S3 use um **VPC Gateway Endpoint** (gratuito).
- Configure **CORS** no API Gateway e no bucket S3 desde o início.

Artefatos obrigatórios no repositório

Arquivo	Conteúdo
specs/spec.md	Problema, personas, histórias com critérios de aceite, como RF-01 a RF-05 se aplicam ao domínio
specs/design.md	Diagrama de arquitetura, modelo ER, rotas da API, Security Groups, ADRs
specs/tasks.md	Tarefas com ID, requisito, responsável e status

Arquivo	Conteúdo
docs/ia-log.md	Prompts importantes → o que a IA gerou → o que vocês aceitaram, rejeitaram ou corrigiram
README.md	Como criar a infraestrutura e publicar o sistema, passo a passo

Cada grupo cria e organiza o próprio repositório. Os arquivos acima podem ter outro nome ou caminho, desde que o README indique onde estão.

Entregas

Aula	O que mostrar
Aula 1 09/10 · em sala	Grupo formado; repositório criado com acesso do professor; domínio escolhido; spec.md inicial gerado com o Copilot (problema, personas, pelo menos 5 histórias, como cada RF se aplica); primeira entrada no ia-log.md. **Link do repositório no Moodle até o fim da aula.**
Aula 2 16/10	spec.md revisado com critérios de aceite; design.md com **diagrama de arquitetura modelado com IA**, modelo ER e rotas da API; ADR do RNF-04; **bucket S3 com uma landing page/protótipo acessível na internet**.
Aula 3 23/10	VPC com subnets pública e privada; **RDS privado** com o schema (scripts SQL versionados); **CRUD do RF-01 funcionando na AWS**, ao vivo; Security Groups explicados; tasks.md atualizado.
Aula 4 30/10	**Upload (RF-02)**, **evento S3 → Lambda → RDS (RF-03)** com log no CloudWatch, **relatório (RF-04)** e front-end publicado; design.md coerente com o que está rodando.
Final 06/11	**ZIP no Moodle** (sem apresentação): código-fonte, scripts SQL, README, pasta specs/, ia-log.md com reflexão final, link do repositório e **evidências** (prints da aplicação e dos recursos AWS) em docs/evidencias/.

Nas Aulas 2 a 4, cada grupo faz uma **demonstração ao vivo de cerca de 5 minutos**, seguida de perguntas. Vale o que está **rodando na AWS** no momento, não apenas o que está no repositório.

Avaliação

Projeto 2 = 50% entregas do grupo + 50% prova individual (27/11).

Entrega	Peso nos 50% do projeto
Aula 1 (09/10)	5%
Aula 2 (16/10)	15%
Aula 3 (23/10)	25%

Entrega	**Peso nos 50% do projeto**
Aula 4 (30/10)	25%
Final (06/11)	30%

- Entrega não apresentada na data pode ser mostrada na aula seguinte valendo **no máximo 50%**.
- Quanto maior o grupo, maior o rigor na correção.
- A nota individual pode ser ajustada para quem não souber explicar o próprio projeto.

Bom trabalho e bom deploy!
Dúvidas técnicas serão esclarecidas em aula.